

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[Subversion Book/WebDAVAnd Autoversioning](#)



[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBook/DeveloperInformation](#) ›

[SubversionBook/Reference](#) ›

[SubversionBook/SubversionForCVSUsers](#) ›

[SubversionBook/CVSRepositoryMigration](#) ›

[SubversionBook/Troubleshooting](#) ›

[SubversionBook/WebDAVAndAutoversioning](#)

Login:

Password:

[Join](#)

"Perl is executable line noise,
Python is executable
pseudo-code."

Appendix A. WebDAV 와 자동 버전화

[barmi](#)
[엄길웅](#)
[DocbookSgml/Mail-User-HOWTO](#)
[YumUsage](#)

Table of Contents

A.1.

A.1. [WebDAV 의 기본적인 개념](#)

A.1.1. [WebDAV](#)

A.1.2. [DeltaV 확장](#)

A.2. [Subversion 와 DeltaV](#)

A.2.1. [Subversion 의 DeltaV에의 매핑](#)

A.2.2. [자동 버전화의 서포트](#)

A.2.3. [mod_dav_lock에 의한 대안](#)

A.3. [자동 버전화의 상호 협조성](#)

A.3.1. [Win32 웹 폴더](#)

A.3.2. [Mac OS X](#)

A.3.3. [Unix: Nautilus 2](#)

A.3.4. [Linux davfs2](#)



kss@kldp.org

A.1.

WebDAV 는 HTTP의 확장으로, 파일 공유를 위한 표준으로서 자리잡아 가고 있습니다. 오늘날의 운영체제들은 극단적으로 웹을 의식하고 있으므로 많은 운영체제가 WebDAV 서버에 의해 공개된 "공유"를 마운트하기 위한 기능을 기본으로 내장하고 있습니다.

만약 Apache/mod_dav_svn 을 Subversion 네트워크 서버로 이용한다면, 아마 WebDAV 서버도 사용하고 있을 것입니다. 이 부록에서는 WebDAV 프로토콜의 성질에 대한 몇가지 배경지식을 제공하고 Subversion이 어떻게 그것을 이용하며, WebDAV를 지원하는 다른 소프트웨어와 어떻게 잘 협조하는지를 다룹니다.

A.1. WebDAV 의 기본적인 개념

이 마디는 WebDAV의 개념에 대해서 아주 간략하고 일반적인 개요를 제공합니다. 이것은 클라이언트와 서버의 사이의 WebDAV 의 호환성에 관한 문제를 이해하기 위한 기초가 됩니다.

A.1.1. WebDAV

RFC 2518 은 몇 가지 개념과 거기에 동반하는 HTTP 1.1 의 확장 메소드를 정의하고 있습니다. 그것은 웹을 좀 더 보편적인 읽고 쓰기 가능한 매체로 만들어주는 것입니다. WebDAV 호환 웹 서버는 일반적인 파일 서버처럼 동작할 수 있고, 클라이언트는 WebDAV의 "공유"를 NFS나 SMB 공유처럼 마운트 할 수 있다는 것이 기본적인 개념입니다.

그러나 RFC 2518 은 DAV라는 말 중에 "V"가 있음에도 불구하고, 어떤 종류의 버전(Version) 관리 모델도 **제공하지 않습니다**. 기본적인 WebDAV 클라이언트와 서버는 파일이나 디렉토리가 하나의 버전만 존재한다고 가정하고, 그것을 계속해서 덮어씁니다. [\[1\]](#)

기본적인 WebDAV에서 도입된 새로운 개념과 메소드는 다음과 같습니다.

새로운 쓰기 메소드

웹 자원을 만들거나 덮어쓰는 표준적인 HTTP PUT메소드에 외에도 WebDAV 에서는 새로운 COPY 와 MOVE 메소드를 정의되어서 자원을 복제하거나 이동하거나 할 수가 있습니다.

컬렉션

이것은 자원(URI)의 그룹을 가리키는 WebDAV 용어입니다. 대부분의 경우 그것은 "디렉토리"라는 말과 같은 의미입니다. "/" 문자로 끝나는 것은 컬렉션이라고 부를 수 있습니다. 파일 자원은 PUT 메소드로 고쳐 쓸 수 있거나 만들어지거나 합니다만, 집합 자원은 새로운 MKCOL 메소드로 만들어집니다.

속성

이것은 Subversion에 나와와 같은 아이디어입니다 파일과 집합에 부수 한 메타데이터입니다. 클라이언트는 새롭다 PROPFIND 메소드를 사용해 자원에 부수 한 속성을 일람표 가리키거나 추출하거나 할 수 있습니다. 그리고, PROPPATCH 메소드를 사용해 변경할 수 있습니다. 몇개의 속성은 완전하게 유저에 의해 만들어지고 제어됩니다(예를 들어, "color"로 불리는 속성), 또 다른 것은 완전하게 WebDAV 서버에 의해 만들어지고 제어됩니다(예를 들어, 파일 의 마지막 수정 시각을 포함한 속성). 최초의 것은 "dead" 속성으로 불려 나머지의 것은 "live" 속성으로 불립니다.

락

WebDAV 서버는 클라이언트에 대한 락의 기능을 준다 일이 생깁니다. 이 기능은 임의입니다. 대부분의 WebDAV 서버는 이 기능을 제공하고 있습니다만. 만약 존재하면, 클라이언트는 새롭다 LOCK 와 UNLOCK메소드를 사용해 자원에의 액세스를 조정 할 수가 있습니다. 대부분의 경우, 이러한 메소드는 배타적인 기입해 락을 만든다 위해(때문에) 이용됩니다(>로 논의했다 같게), 공유 기

입 락도 가능한 것은 않습니다만.

A.1.2. DeltaV 확장

RFC 2518 은 버전관리라는 개념이 없기 때문에, WebDAV에 버전 관리 기능을 추가한 RFC 3253을 쓰는 일이 다른 그룹에 맡겨졌습니다. WebDAV/DeltaV 클라이언트와 서버는 자주 "DeltaV" 클라이언트와 서버로 불립니다. DeltaV는 기본적인 WebDAV의 존재를 포함하고 있기 때문입니다.

DeltaV 는 완전히 새로운 단어를 도입했습니다만, 놀라지 마세요. 생각은 매우 직관적입니다. DeltaV로 도입된 새로운 개념과 메소드는:

자원 마다의 버전화

CVS나 다른 버전 관리 시스템과 같이 DeltaV는 각각의 자원이 무한한 개수의 상태를 가질 수 있다고 가정합니다. 클라이언트는 VERSION-CONTROL라는 새로운 메소드를 사용해 자원을 버전 관리 상태로 둡니다. 이것은 새 버전 관리되는 자원(VCR)을 만듭니다. VCR을 변경할 때, (PUT나PROPPATCH등의 메소드를 이용해서), 자원의 새로운 상태가 작성됩니다. 이것을 버전 자원(VR)이라고 부릅니다. VCR와 VR도 보통 URL에 의해 정의되는 웹 자원입니다. VR는 사람이 알기 쉬운 이름도 가질 수 있습니다.

서버측 작업본 모델

몇몇 DeltaV 서버는 가상적인"작업 스페이스"를 서버상에 만드는 능력을 서포트하고 있습니다. 거기서 모든 작업이 실행됩니다. 클라이언트는MKWORKSPACE 메소드를 사용해 사적인 영역을 만들어, 그리고 특정의 VCR 를 작업 스페이스에"체크아웃" 하는 것으로 변경하고 싶다 그렇다고 하는 것을 나타내, 그것들을 편집해, 그리고 한번 더 "체크인"합니다. HTTP의 말에서는, 메소드의 흐름으로서는, CHECKOUT, PUT, CHECKIN 됩니다. 각각의 CHECKIN의 다음에, 새로운 VR 가 만들어져 편집된 VCR 의 내용은 최신의 VR를"지시하는 것" 같게 됩니다. 각각의 VCR 는 또"이력"자원을 가져, 다양한 VR상태를 기록해, 문의를 합니다.

클라이언트측 작업 카피 모델

몇개의 DeltaV 서버는, 클라이언트는 특정의 VR에 의한 프라이빗 작업 카피를 가질 수가 있다고 하는 생각을 서포트합니다. 이것은 CVS와 Subversion가 동작하는 방식입니다) 클라이언트가 서버에 별로 수정을 위탁하고 싶은 경우, 그것은 처음에 일시적인 서버 호랑이 자크션을MKACTIVITY 메소드에 의해 만듭니다 (액티버티로 불립니다). 그리고 클라이언트는 변경하고 싶으면 생각하는 VR 각각의 위에서CHECKOUT 를 실행해, 그것은 몇개의 일시적인"작업 자원"을 액티버티안에 만듭니다. 그리고 그것은PUT 와 PROPPATCH메소드를 사용해 수정할 수가 있습니다. 마지막으로, 클라이언트는 작업 자원 각각의 위에서 CHECKIN 를 실행해, 그것은 각각의 VCR 내부에 새로운 VR를 만들어, 그리고 액티버티 전체가 삭제됩니다.

설정

DeltaV 는"설정"으로 불리는 VCR의 유연한 모임을 정의 합니다

만, 그것은 특정의 디렉토리에 응답할 필요는 없습니다. VCR의 내용의 각각은 UPDATE 메소드를 사용해 특정의 VR를 지시할 수가 있습니다. 한번 설정이 완전하게 되면, 클라이언트는 설정 전체의 "snapshot" 를 만들 수가 있습니다. 이것은 "baseline"로 불립니다. 클라이언트는 CHECKOUT 와 CHECKIN 메소드를 사용해 설정의 특정 상태를 잡습니다. 정확히, 그러한 메소드를 VCR의 특정의 VR 상태를 만들기 위해서(때문에) 사용하는데 답았습니다.

확장성

DeltaV 는 새로운 메소드 REPORT를 정의합니다만 그것은 클라이언트와 서버가 독자 데이터 교환을 실행하는 것을 허락한다 물건입니다. 클라이언트는 REPORT 요구를 독자적인 데이터가 있는 속성 라벨이 붙은 XML의 보디를 이라고도 되어 송신합니다. 서버가 이 특정의 리포트형을 이해할 수 있는 것을 가정해, 그것은 역시 독자적인 XML 보디를 응답합니다. 이 기술은 XML-RPC와 잘 답았습니다.

자동 버전화

대부분의 사람에게 있어, 이것은 DeltaV의 "가장 매력적인" 기능입니다. 만약 DeltaV 서버인가 이 기능을 서포트하고 있으면, 기본적인 WebDAV 클라이언트(즉 버전화를 의식하지 않는 클라이언트)는 또한 서버에 기입할 수가 있어 그 서버는 어쨌든 입다물어 버전화 처리를 실행합니다. 제일 간단한 예에서는, 기본적인 WebDAV 클라이언트로부터의 무지한 PUT 는 서버에 의해 CHECKOUT, PUT, CHECKIN의 편성이라고 해석됩니다.

A.2. Subversion 와 DeltaV

그럼, Subversion는 다른 DeltaV 소프트웨어와 어떠한 "호환성" 이 있는 것일까요? 간단하게 말하면(자): 그만큼. 적어도, 아직 그렇게 호환성은 없고, Subversion 1.0 에서도 그렇겠지요.

libsvn_ra_dav 는 DeltaV 요구를 서버에 송신합니다만, Subversion 클라이언트는 일반적인 DeltaV 클라이언트가 **아닙니다**. 실제, 그것은 서버에 몇개의 독자적인 기능을 기대하고 있습니다(특히, REPORT요구와 같은 것을 통해서). 한층 더 mod_dav_svn 는 일반적인 목적의 DeltaV 서버가 **아닙니다**. 그것은 단지 DeltaV 정의의 한정된 일부를 실장하고 있는에 지나지 않습니다. 좀 더 일반적인 WebDAV나 DeltaV 클라이언트는 그것과 매우 잘 협조 동작할 수가 있을지도 모르지 않습니다만, 그것은 그 클라이언트 하지만 서버가 실장하고 있는 한정된 기능과 일치하고 있는 것 같은 조작(을)를 할 때 뿐입니다. Subversion 개발 팀은 Subversion의 향후의 릴리스 그리고 일반적인 WebDAV와의 상호 운용을 취할 계획이 있습니다.

A.2.1. Subversion 의 DeltaV에의 매핑

다양한 Subversion 클라이언트의 조작이 DeltaV를 사용하는 방법의 매우 "하이레벨의" 방식을 들어 둡니다. 대부분의 경우, 이러한 설명은 단순화 너무 하고 있습니다. Subversion의 원시 코드를 읽거나 개발자에게 (듣)묻거나 하는 것의 대신에 된다고는 **생각하지 말아주세요**.

svn checkout/list

직접적인 아이의 일람을 취득하기 위해서, 집합 후에 깊이 1 의 PROPFIND 를 실행합니다. 각각의 아이에게, GET (혹은 PROPFIND)를 실행합니다. 집합 중(안)에서 재귀 해, 이것을 반복합니다.

svn commit

MKACTIVITY로 액티버티를 만들어, 변경되었다 아이템 마다 CHECKOUT 를 실행해, 새로운 데이터 (을)를PUT 해, 마지막에MERGE 요구를 해, 그것이 암묵에 호출하는CHECKIN 가 모든 작업 자원에 대해서 실행됩니다.

svn update/switch/status/merge/diff

작업 카피의 혼합 버전(혼합 URL) 상태를 나타내는 독자적인 REPORT 요구를 보냅니다. 서버는 어느 아이템에 갱신이 필요한가를 나타내는 독자적인 응답을 보냅니다. 클라이언트는 응답 마다 루프 해, 필요한 GET 와 PROPFIND요구를 실행합니다. update 와 switch 커멘드에서는, 작업 카피에 새롭다 데이터를 배치합니다. diff 와 merge 커멘드에서는 데이터를 작업 카피 (와)과 비교해, 로컬인 수정에 대한 변경을 적용하는 일도 있습니다.

A.2.2. 자동 버전화의 서포트

집필 시점에서, 이 세상에게는 아직 매우 몇 안 되는 종류의 DeltaV 클라이언트 밖에 없다고 하는 것이 현실입니다. RFC 3253 은 아직 비교적 새로운 규격입니다. 그러나, 유저는"범용적인"클라이언트에 액세스 한다 (이)가 할 수 있습니다. 그것은, 거의 모든 현대적인 오퍼레이팅 (operating) 시스템은, 통합된 기본적인 WebDAV 클라이언트를 실장하고 있기 때문입니다. 이것을 마음에 세워, Subversion 개발자는 만약 Subversion1.0이 어떠한 제휴 기능을 가지게 된다고 해도, DeltaV의 자동 버전화의 구조를 서포트하는 것이 가장 좋은 선택 (이)라고 생각하고 있습니다.

mod_dav_svn 의 자동 버전화의 구조를 유효하게 하려면 , httpd.conf 파일의 Location블록으로, SVNAutoversioning 명령을 사용해 주세요. 이런 느낌입니다:

```
Location /repos
    DAV svn
    SVNPath /absolute/path/to/repository
    SVNAutoversioning on
/Location
```

보통, WebDAV 클라이언트가 저장소(repository)의 장소에 있는 패스에 PUT 를 발행하면(자), mod_dav_svn는 그 요구를 무조건 거부합니다. (그것은 보통은, DeltaV"액티버티"에 있다 "작업 자원"상에 그러한 조작을 허락할 뿐입니다). 그러나SVNAutoversioning 를 유효하게 하면 (자), 그 서버는PUT 요구를 횡령해, 내부적으로는 MKACTIVITY, CHECKOUT, PUT, CHECKIN의 일련의 명령으로서 처리합니다. 일반적인 로 그 메시지가 자동적으로 생성되어 새로운 파일 시스템 리버전이 만들어집니다.

매우 많은 operating system가 벌써 통합되었다 WebDAV 능력을 가지고 있으므로, 이 기능의 이용 형태는 매우 훌륭하다 물건이 됩니다: Microsoft Windows 나 Mac OS 를 실행하고 있는 보통 유저가 있는 오피스를 상상해 보세요. 각각의 컴퓨터 (은)는 Subversion의 저장소(repository)를 "마운트" 하고 있어, 그것은 보통 네트워크 공유로 보입니다. 그들은 평상시 하고 있도록(듯이) 서버를 사용하겠지요: 서버의 파일을 열어, 편집해, 서버 써 되돌립니다. 그러나, 이 이야기 중(안)에서는, 서버는 모든 것을 자동적으로 버전화합니다. 나중에, 시스템 관리 책임자가 Subversion 클라이언트 (을)를 사용해, 모든 낡은 버전을 검색해, 추출할 수가 있습니다.

이 이야기는, 현실로 할 수 있는 것일까요? 아니, 아직 완전하게는. 제일의 장애는 Subversion 1.0 은 WebDAV의 LOCK (이)나 UNLOCK 메소드를 서포트하고 있지 않다고 한다 일입니다. 대부분의 operating system의 DAV 클라이언트는 DAV의 형태로 마운트된 네트워크 공유로부터 직접 열린 자원에 대해 LOCK 하려고 합니다. 현재, 유저는 DAV 공유로부터 파일을 로컬 디스크에 카피해, 편집해, 그리고 한번 더, 서버에 써 되돌리지 않으면 안됩니다. 이상적인 자동 버전화는 할 수 없습니다만, 그런데도, 작업할 수 있습니다.

A.2.3. mod_dav_lock에 의한 대안

mod_dav Apache 모듈은 복잡한 모듈입니다: 그것은 모든 WebDAV 와 DeltaV 메소드를 이해해 해석하는 것이 할 수 있습니다. 그러나, 아직 자원 자신에게 액세스 하려면 연구 최종 단계의 "공급 모듈" 에 의존하고 있습니다.

제일 간단한 경우로서는, 유저는 mod_dav_fs 를 mod_dav의 공급 모듈로서 이용할 수가 있습니다. mod_dav_fs 는 통상의 파일 시스템을 파일이나 디렉토리를 격납하는 장소에 이용 해, 기본적인 WebDAV 메소드만을 이해합니다. DeltaV는 이해 성과 선.

한편 Subversion는 mod_dav_svn 를 mod_dav 의 공급 모듈로서 사용합니다. mod_dav_svn는 LOCK를 들여다 보았다 모든 WebDAV 메소드를 이해해, DeltaV 메소드의 상당한 부분을 이해 합니다. 그것은 진짜 파일 시스템에 있다고 하는 것보다도, Subversion 저장소(repository)에 있는 데이터에 액세스 합니다. Subversion 1.0 은 락을 서포트합니다만, 그것은 Subversion가 카피·수정·머지 모델을 사용하고 있는 것으로, 실제로 실장하는 것이 매우 어렵기 때문입니다. [\[2\]](#)

Apache httpd-2.0 으로, mod_dav 는, 공급 모듈은 그것을 받아들이는 뜻이 있다고 하는 전제 앞으로, 사적인 데이터베이스에 걸치는 락을 기록하는 것에 의해 LOCK 메소드를 서포트합니다. 그러나 Apache httpd-2.1 이후에서는 이 락의 서포트는 독립한 모듈 mod_dav_lock 에 분할 됩니다. 이 모듈은 어떠한 mod_dav 공급 모듈에도 락 데이터베이스의 이용을 인정해 거기에는 mod_dav_svn 도 포함됩니다. mod_dav_svn는 실제로는 락을 이해하지 않음에도 불구하고, 입니다.

아직 혼란합니까?

요컨데, Apache httpd-2.1(로 그 이후)에서는, mod_dav_lock를 사용해, mod_dav_svn 가 제대로 LOCK 요구를 내고 있다고 하는 체 을 하는 것이 할 수 있습니다. mod_dav_lock 가 httpd 에 짜넣어 컴파일 되고 있을까 httpd.conf로 로드 되고 있는 것을 확인해 주세요. 그리고 Location 에 DAVGenericLockDB 명령을 추가해 주세요. 이하와 같은 느

깁입니다:

```
Location /repos
  DAV svn
  SVNPath /absolute/path/to/repository
  SVNAutoversioning on
  DavGenericLockDB /path/to/store/locks
/Location
```

이 테크닉은, 조금 위험합니다: 어느 의미, mod_dav_svn 는 WebDAV 클라이언트에 거짓말을 하고 있는 것입니다. LOCK 요구를 받고 넣는 체를 합니다만, 실제로는 어떤 레벨에 대해서도 락은 강제당하고 있지 않습니다. 만약 제2의 WebDAV 클라이언트가 같은 자원 에 대해서 LOCK 하려고 하면(자), mod_dav_lock는 그것을 알아, 요구를 적절히 거부합니다. 그러나, 보통 Subversion 클라이언트가 보통 **svn commit**를 통해서 하는 파일 변경은, 전혀 막을 수 없습니다!. 만약 이 테크닉을 사용한다면, 유저에게, 다른 사람의 변경을 어렵잡아 망칠 기회를 주어 버립니다. 특히, WebDAV 클라이언트는 통상의 svn 클라이언트에 의해 위탁되었다 수정을 틀려 덧쓰기해 버릴지도 모릅니다.

한편, 충분히 주의해 환경을 설정하면, 위험을 줄일 수가 있습니다. 예를 들어, 만약 유저의 **모든 것** 이(svn 클라이언트 (은)는 아니게) 기본적인 WebDAV 클라이언트를 통해서 작업하고 있다면, 이야기는 잘 되겠지요.

A.3. 자동 버전화의 상호 협조성

이 마디에서는, 가장 일반적인 WebDAV 클라이언트(현시점에서의)와 그것이 SVNAutoversioning 명령을 사용하는 mod_dav_svn 서버에 대하는 조작을 어떤 느껴에 잘 하는지를 설명합니다. RFC 2518 은 조금 크고, 아마, 너무 유연한 규격입니다. WebDAV 클라이언트 의 각각은 푼푼이 다른 움직임을 해, 각각 조금씩 다른 문제를 일으킵니다.

A.3.1. Win32 웹 폴더

Windows 98, 2000, 이라고 XP는, "웹 폴더"로서 알려져 있는 통합된 WebDAV 클라이언트를 실장하고 있습니다. Windows 98에서는, 그 기능은 명시적으로 인스톨 할 필요가 있습니다. 만약 존재하고 있으면, "웹 폴더" 는 마이 컴퓨터 의 안에, 직접 보일 것입니다. Windows 2000 으로 XP에서는, 단지 마이 네트워크의 곳을 열어, 네트워크 플레이스의 추가의 아이콘을 실행할 뿐입니다. 다이얼로그가 나오면(자), WebDAV 의 URL를 입력합니다. 공유 폴더는 마이 네트워크 플레이스에 표시되게 됩니다.

대부분의 기입 조작은 자동 버전화 된 mod_dav_svn 서버에 잘 액세스 할 수 있습니다만, 몇개의 문제도 있습니다:

- 만약 컴퓨터가 NT도메인의 멤버라고, WebDAV 공유에 접속할 수 없는 것처럼 보입니다. 그것은 반복해 이름과 패스워드를 요구해, 그것은 Apache 서버가 인증을 요구하지 않을 때조차 일어납니다. 어느 사람들은, 웹 폴더는 Microsoft의 SharePoint

DAV 서버에 별로 동작하도록(듯이) 설계되었기 때문이라고 생각하고 있습니다. 만약 머신이 NT도메인이 일부에서 없으면, 그 공유는 문제 없게 마운트 할 수 있습니다. 이 수수께끼는 아직도 해결하고 있지 않습니다.

- 파일은 공유로부터 편집을 위해서(때문에) 직접 열 수가 없습니다. 항상 읽어들이 전용이 됩니다. mod_dav_lock의 테크닉은 도움에 되지 않습니다. 웹 폴더는 LOCK 메소드를 전혀 이용하지 않기 때문입니다. 그러나, 전에 지정한 것처럼 "카피, 편집, 써 반환" 방법은 잘 움직입니다. 공유상의 파일은 로컬로 편집한 카피로 문제 없게 덧쓰기할 수 있습니다.

A.3.2. Mac OS X

애플의 OS X operating system는 통합된 WebDAV 클라이언트입니다. 파인더의 Go 메뉴로 "서버에 접속" 아이템 (을)를 선택해 주세요. WebDAV의 URL를 입력하면, 보통 파일 서버와 같이 데스크탑상의 디스크로서 보이게 됩니다. [\[3\]](#)

불행하게도, 이 클라이언트는 LOCK의 서포트가 없기 때문에, mod_dav_svn의 자동 버전화의 동작을 거부합니다. Mac OS X는 최초의 HTTP OPTIONS 기능 교환시에 LOCK의 능력이 없는 것을 알아, Subversion 저장소(repository)를 읽어들이 전용의 공유로서 마운트하는 결정을 합니다. 그 후에는 기입 조작은 전혀 할 수 없습니다. 읽고 쓰기 가능한 공유 (으)로서 저장소(repository)를 마운트하기 위해서는, 전에 논의한 mod_dav_lock의 트릭을 사용할 필요가 있습니다. 한번 락이 움직이고 있는 것처럼 보이면, 공유는 매우 잘 행동합니다. 파일은 읽고 쓰기 모드로 직접 열 수가 있습니다. 다만, 보존의 조작은, 실제로는 일시적인 장소에 PUT 해, 원래의 파일을 DELETE 해, 그 후, 원의 파일명칭 에, 그 일시파일을 MOVE 하는 것으로 행합니다. 보존마다, 새로운 Subversion의 리버전을 세 개 할 수 있습니다.

또 하나의 주의: OS X의 WebDAV 클라이언트는 HTTP 김다이렉트에 대해 필요이상으로 민감합니다. 만약 저장소(repository)를 전혀 마운트할 수 없다 (이)라면, httpd.conf의 BrowserMatch 명령을 유효하게 할 필요가 있을지도 모릅니다:

```
BrowserMatch "^WebDAVFS/1. [012]" redirect-carefully
```

A.3.3. Unix: Nautilus 2

Nautilus는 GNOME 데스크탑의 공식적인 파일 관리자/브라우저입니다. 홈 페이지는 <http://www.gnome.org/projects/nautilus/>에 있습니다. Nautilus 윈도우로 WebDAV URL를 입력하는 것만으로, DAV 공유가 로컬 파일 시스템과 같이 보입니다.

일반적으로, Nautilus 2는 mod_dav_svn의 자동 버전화가 거의 잘 움직이지만, 이하에는 주의입니다:

- 공유로부터 직접 연 파일은 읽어들이 전용으로 다루어집니다. mod_dav_lock의 트릭은 전혀 효과가 없는 것처럼 보입니다. Nautilus는 LOCK 메소드를 완전히 문제에 하고 있지 않는 것 같

습니다. "로컬에 카피, 편집, 써 반환" 의 트릭은 움직이는 것 같습니다만. 불행하게도 Nautilus 는 DELETE 를 사용해 최초로 낡은 파일을 뺏쓰기하므로, 그것이, 하나 더의 리버전을 만들어 버립니다.

- 파일을 뺏쓰기하거나 작성할 경우에는, Nautilus 는 우선 하늘의 파일을PUT 하고 나서, 제2의PUT로 그것을 뺏쓰기합니다. 이것은, 하나가 아니고, 두 Subversion 파일 시스템 리버전을 만들어 버립니다.
- 집합을 삭제할 경우에는, 집합 자신의 대신에, 개별의 아이에 대해서 HTTP DELETE 를 발행합니다. 이것은, 파일의 수만의 새로운 리버전을 만들어 버립니다.

A.3.4. Linux davfs2

Linux davfs2 는 Linux 커널의 파일 시스템 모듈로, 개발 홈 페이지는 <http://dav.sourceforge.net/> 에 있습니다. 인스톨 하면(자), WebDAV 네트워크 공유가 보통 Unix의**mount** 커멘드로 마운트할 수 있다 같게 됩니다.

소문에 의하면, 이 DAV 클라이언트는 mod_dav_svn의 자동 버전 화가 전혀 동작하지 않다는 것입니다. 서버에 대한 기입의 모두에 대해, 우선LOCK 요구를 냅니다만, 이것은 mod_dav_svn 가 서포트하고 있지 않는 것입니다. 현시점에서는 mod_dav_lock 가 이 문제를 해결하는지 어떤지를 나타내는 자료는 없습니다.

Notes

[1]

이런 이유 때문에 사람들은 농담으로 WebDAV 클라이언트를 "WebDA" 클라이언트라고 부르기도 합니다!

[2]

Subversion는 언제의 날인가, 보존된 체크아웃 락의 모델을 개발할지도 모릅니다. 그것은 카피·수정·머지 모델과 잘 공존할 수가 있습니다. 하지만, 아마 곧바로는 무리이겠지요.

[3]

Unix 유저는**mount -t webdav URL /mountpoint** (을)를 실행할 수가 있습니다.

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

W3C XHTML 1.0

W3C CSS

POWERED BY

Last modified: 2004-12-17 14:20:49, Processing time: 0.0363 sec

[IE7 다운로드](#)

이제 Google에 최적화. 지금 향상된 보안 기능 을 사용. 무료!

[레노버 쇼핑몰](#)

인텔(R) 센트리노(R) 듀오 기반 V100. PC탄생 25주년 이벤트